API: ESERCITAZIONI

STRUTTURE DATI V

RED-BLACK TREES

Gabriele Paveri-Fontana

Mail: gabriele.paveri@polimi.it

Recap RBT: Red-Black Trees

- Un *albero rosso-nero* (red-black tree, RBT) T è un BST tale che:
  - Ogni nodo x di T ha un attributo aggiuntivo $x.color$.
  - T ha un attributo aggiuntivo $T.nil$, che è un nodo sentinella (senza chiave): tutti i puntatori a NIL sono sostituiti con puntatori a $T.nil$.
    - ✓ I nodi diversi da $T.nil$ sono detti *nodi interni*.
    - ✓ $T.nil$ è detto *nodo esterno*.
  - Valgono le seguenti proprietà (dette *proprietà RB*):
    1. Ogni nodo è rosso o nero ($x.color = RED$ oppure $x.color = BLACK$)
    2. La radice è nera ($T.root.color = BLACK$)
    3. Il nodo esterno è nero ($T.nil.color = BLACK$)
    4. I figli di un nodo rosso sono entrambi neri
    5. Per ogni nodo x tutti i cammini da x a $T.nil$ contengono lo stesso numero $bh(x)$ di nodi neri
       - ✓ $bh(x)$ è l'*altezza nera* (black height) del nodo x ($bh(T) \coloneqq bh(T.root)$).
       - ✓ Il nodo x non è contato in $bh(x)$ anche se nero, mentre $T.nil$ sì.

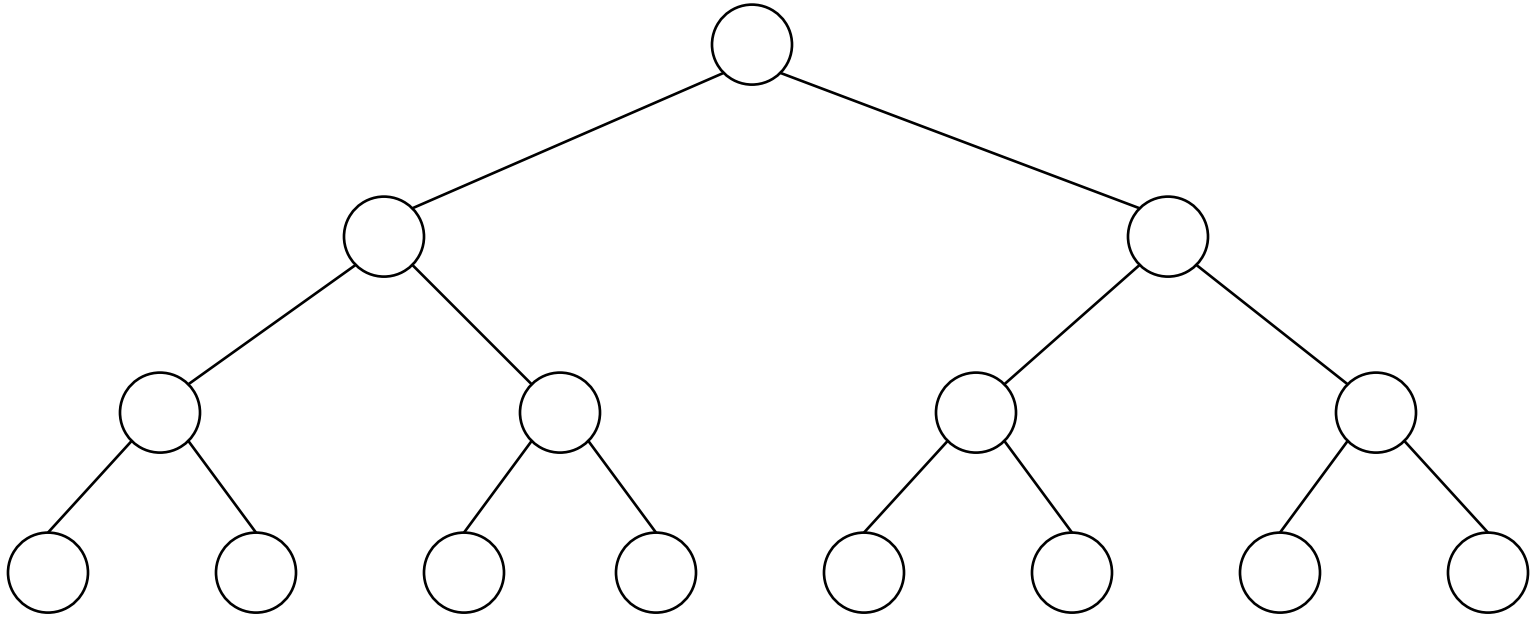
Esempio

- Si consideri un BST completo T di 15 elementi (il valore delle chiavi è irrilevante). Aggiungere il nodo $T.nil$ e colorare i nodi in modo da rendere T un RBT di altezza nera:

a) $bh(T) = 4$
b) $bh(T) = 3$
c) $bh(T) = 2$

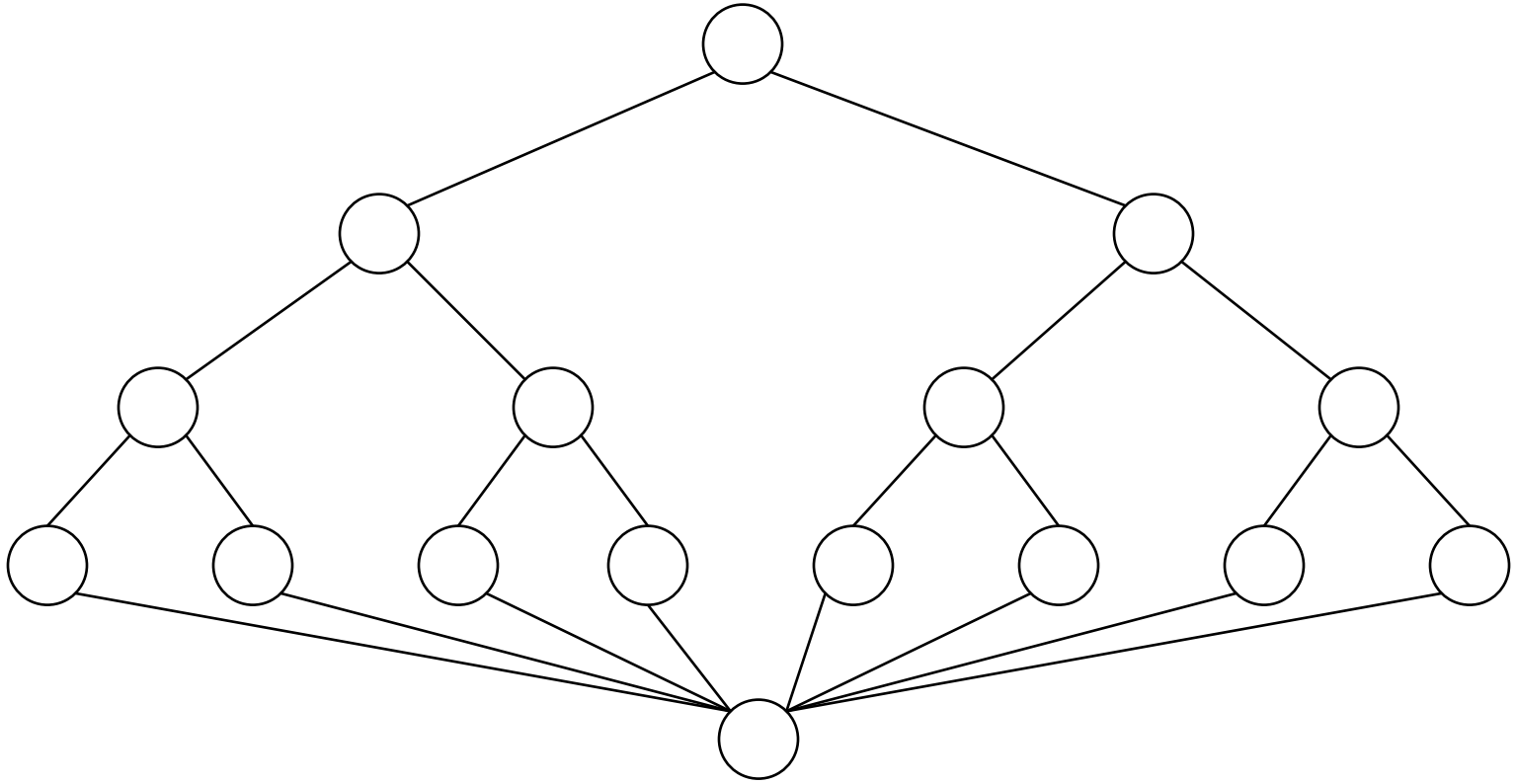
Esempio

- Si consideri un BST completo T di 15 elementi (il valore delle chiavi è irrilevante)…



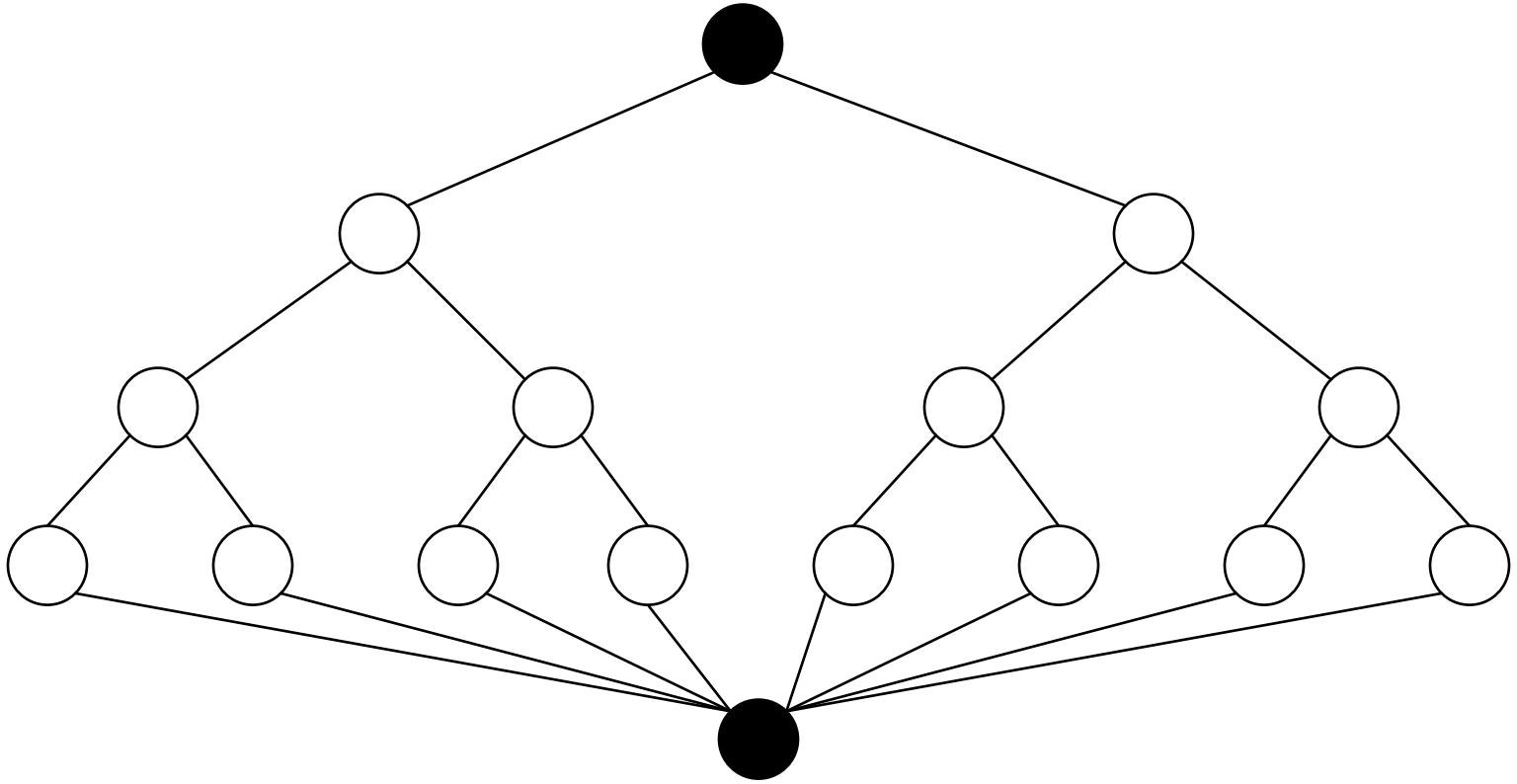
Esempio

- Si consideri un BST completo T di 15 elementi (il valore delle chiavi è irrilevante). Aggiungere il nodo $T.nil$…



Esempio

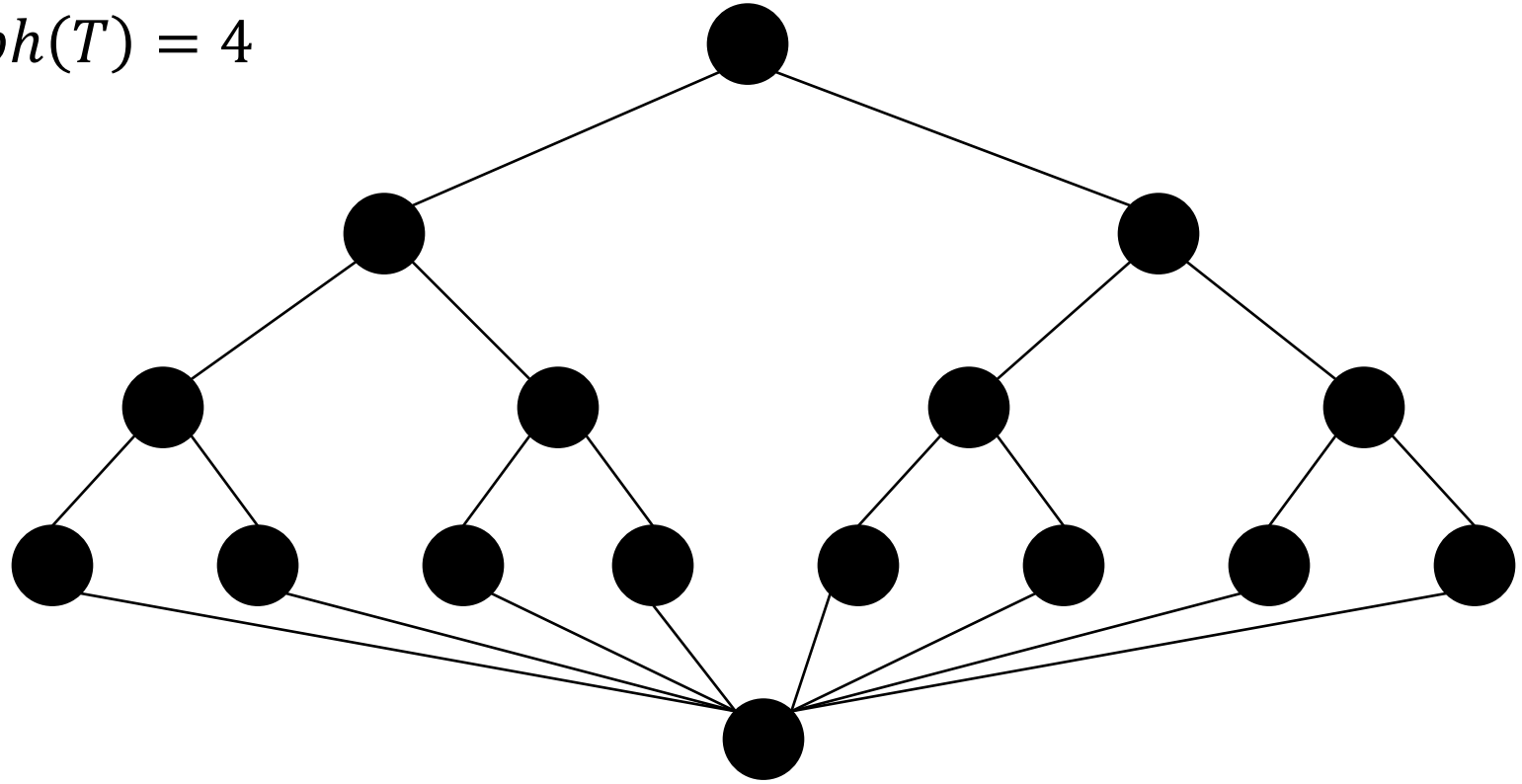
- Si consideri un BST completo T di 15 elementi (il valore delle chiavi è irrilevante). Aggiungere il nodo $T.nil$ e colorare i nodi in modo da rendere T un RBT…



Esempio

- Si consideri un BST completo T di 15 elementi (il valore delle chiavi è irrilevante). Aggiungere il nodo $T.nil$ e colorare i nodi in modo da rendere T un RBT di altezza nera:

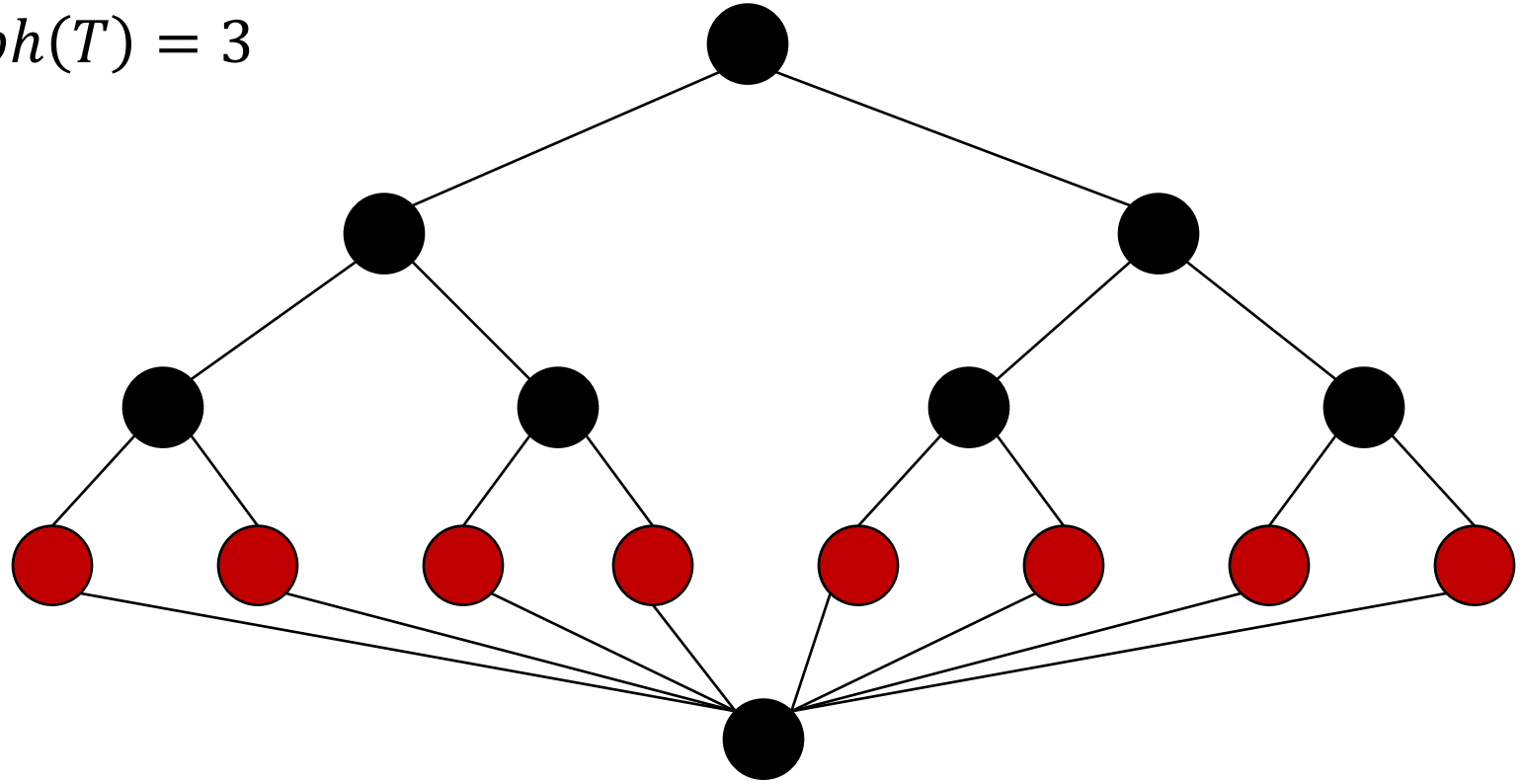
a) $bh(T) = 4$



Esempio

- Si consideri un BST completo T di 15 elementi (il valore delle chiavi è irrilevante). Aggiungere il nodo $T.nil$ e colorare i nodi in modo da rendere T un RBT di altezza nera:

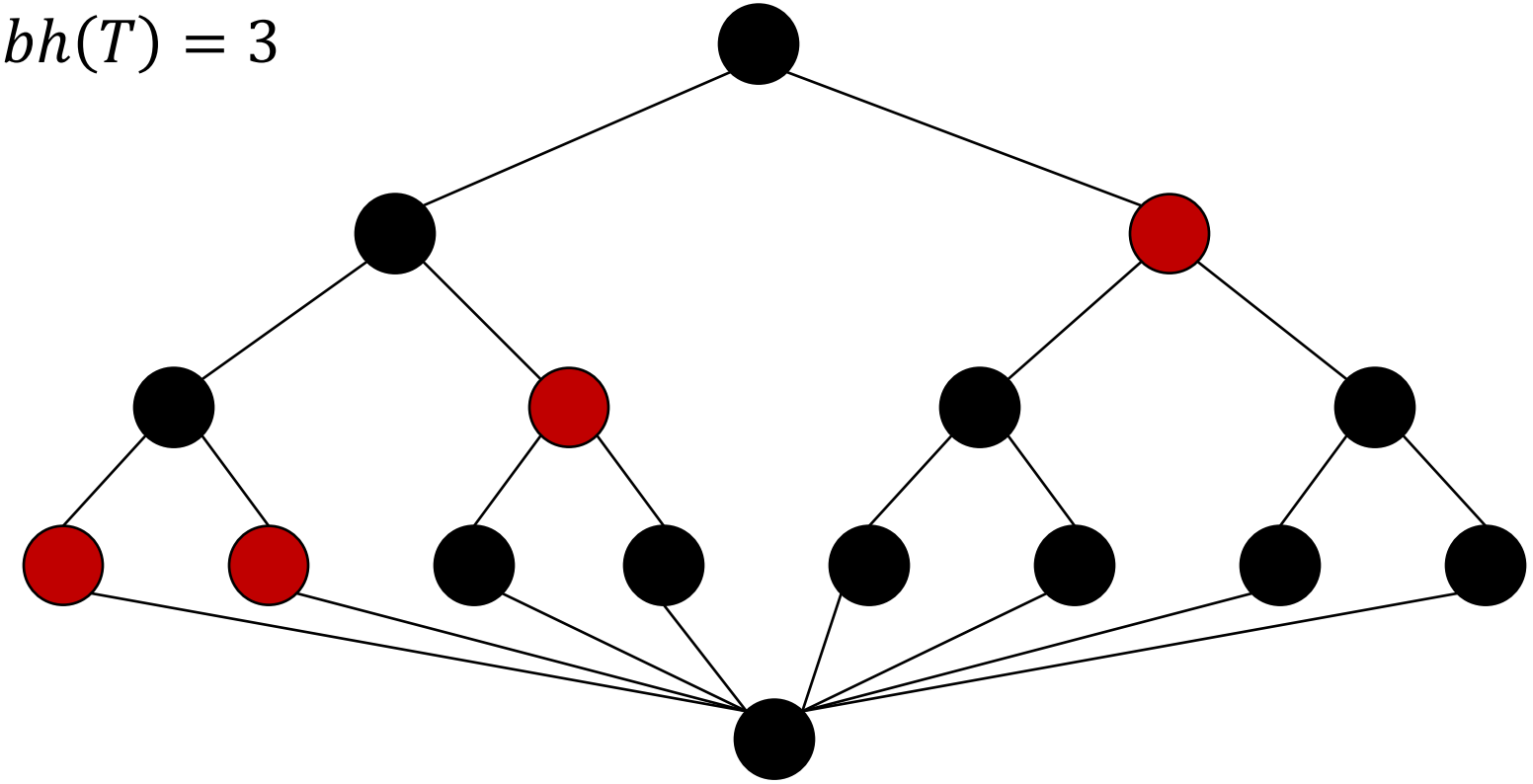
b) $bh(T) = 3$



Esempio

- Si consideri un BST completo T di 15 elementi (il valore delle chiavi è irrilevante). Aggiungere il nodo $T.nil$ e colorare i nodi in modo da rendere T un RBT di altezza nera:

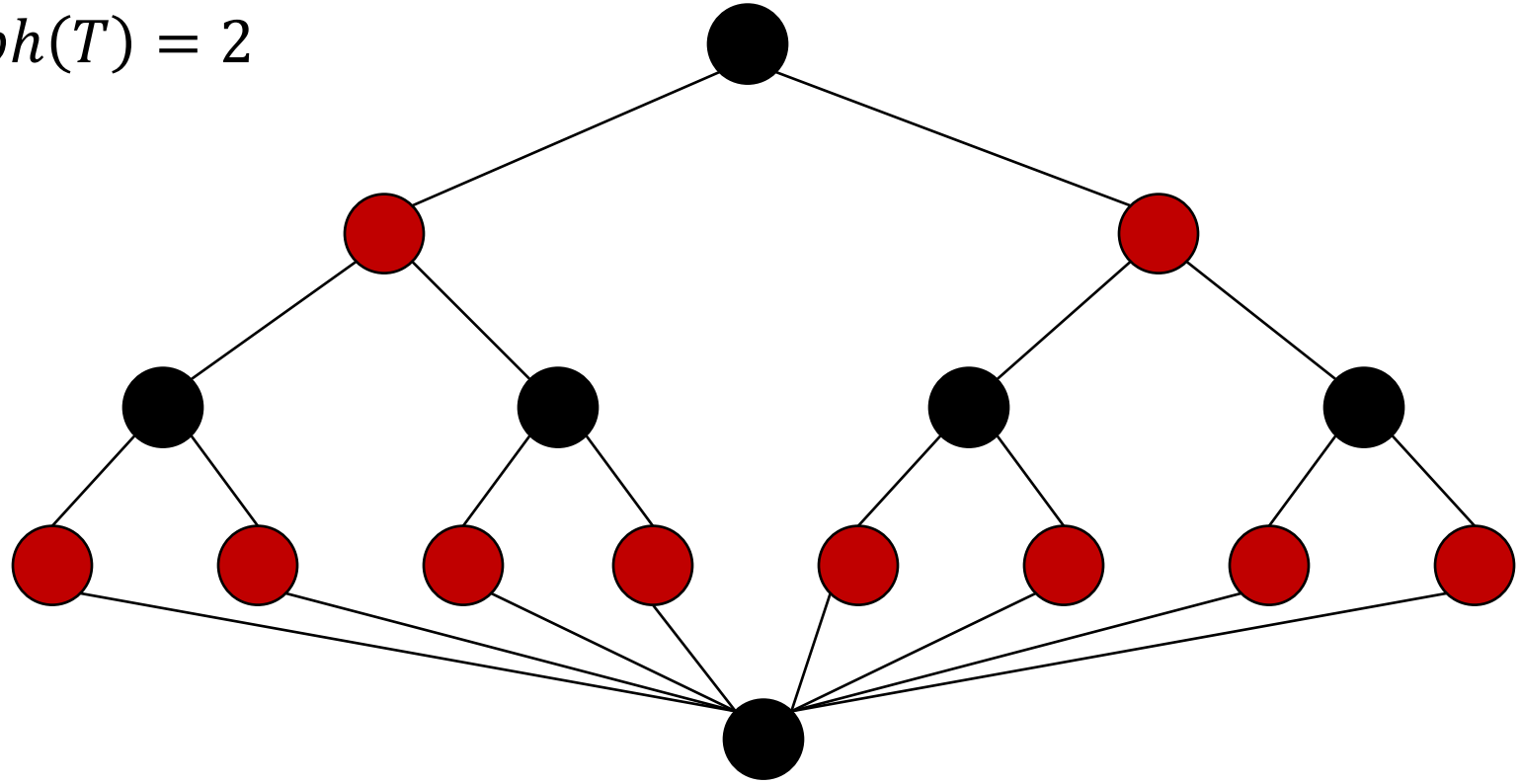
b') $bh(T) = 3$



Esempio

- Si consideri un BST completo T di 15 elementi (il valore delle chiavi è irrilevante). Aggiungere il nodo $T.nil$ e colorare i nodi in modo da rendere T un RBT di altezza nera:

b) $bh(T) = 2$



Recap RBT: Proprietà

- In un RBT T, presi due cammini da un qualunque suo nodo x a $T.nil$, il rapporto tra le loro lunghezze non può superare 2.
  - Infatti ogni cammino da x a $T.nil$ ha esattamente $bh(x)$ nodi neri, dunque il cammino più corto possibile da x a $T.nil$ è lungo $bh(x)$, e la proprietà RB4 garantisce inoltre che il cammino più lungo possibile non superi $2 * bh(x)$.
- Viceversa, se un BST T (in cui tutti i puntatori a NIL siano sostituiti con puntatori a $T.nil$) è tale che, presi due cammini da un qualunque suo nodo x a $T.nil$, il rapporto tra le loro lunghezze non supera mai 2, allora T può essere "colorato" in modo da diventare un RBT.
  - Infatti in tal caso è sempre possibile colorare i nodi in modo da evitare due nodi rossi consecutivi (RB4) e da garantire che per ogni nodo x tutti i cammini da x a $T.nil$ contengano lo stesso numero $bh(x)$ di nodi neri (RB5).

Recap RBT: Proprietà

- Per ogni RBT T, si ha $h/2 \leq bh(T.root) \leq h$.
  - Ovviamente $bh(T.root)$ non può superare l'altezza h di T.
  - Può succedere che $bh(T.root) = h$: basta che T sia completo e che tutti i nodi siano neri (il che rispetta le proprietà RB).
  - La proprietà RB4 garantisce che i figli di ogni nodo rosso siano entrambi neri, dunque non possono mai esserci due nodi rossi di fila lungo un cammino qualsiasi da un nodo verso la foglia $T.nil$. Questo significa che $bh(T.root) \geq h/2$.

Recap RBT: Proprietà

- Per ogni RBT T, si ha $h/2 \leq bh(T.root) \leq h$.
- Per ogni nodo x di un RBT T, $n(x) \geq 2^{bh(x)} - 1$ ($n(x)$ è il numero di nodi interni del sottoalbero con radice x).
  - Dimostriamolo **per induzione** sull'altezza h_x di x.
    - Passo base: x tale che $h_x = 0$
      - ✓ Se $h_x = 0$, allora $x = T.nil$
      - ✓ $bh(T.nil) = 0$
      - ✓ $n(T.nil) = 0 = 2^0 - 1 = 2^{bh(T.nil)} - 1$

Recap RBT: Proprietà

- Per ogni RBT T, si ha $h/2 \leq bh(T.root) \leq h$.
- Per ogni nodo x di un RBT T, $n(x) \geq 2^{bh(x)} - 1$ ($n(x)$ è il numero di nodi interni del sottoalbero con radice x).

  ➢ Dimostriamolo **per induzione** sull'altezza h_x di x.

  - Passo induttivo: assumendo che la tesi valga per tutti i nodi di altezza minore di h_x, dimostriamola per x (con $h_x > 0$).

    ✓ Sia y un nodo figlio di x.
    ✓ Se y è rosso, allora $bh(y) = bh(x)$, altrimenti $bh(y) = bh(x) - 1$.
    In ogni caso $bh(y) \geq bh(x) - 1$.
    ✓ Ma $h(y) < h(x)$, dunque per ipotesi induttiva $n(y) \geq 2^{bh(x)-1} - 1$.
    ✓ Segue che:

$$n(x) = n\,(x.left) + n(x.right) + 1 \geq \left(2^{bh(x)-1} - 1\right) + \left(2^{bh(x)-1} - 1\right) + 1 =$$
$$= 2^{bh(x)} - 1$$

Recap RBT: Proprietà

- Per ogni RBT T, si ha $h/2 \leq bh(T.root) \leq h$.
- Per ogni nodo x di un RBT T, $n(x) \geq 2^{bh(x)} - 1$ ($n(x)$ è il numero di nodi interni del sottoalbero con radice x).
- Un RBT T con n nodi interni ha altezza $h \leq 2\log_2(n+1)$.
  - T ha almeno $2^{bh(T.root)} - 1$ nodi interni
  - Ma $bh(T.root) \geq h/2$
  - Segue che

$$n \geq 2^{bh(T.root)} - 1 \geq 2^{h/2} - 1 \Rightarrow$$

$$\Rightarrow 2^{h/2} \leq n + 1 \Rightarrow$$

$$\Rightarrow h \leq 2 * \log_2(n+1)$$

Recap RBT: Proprietà

- Per ogni RBT T, si ha $h/2 \leq bh(T.root) \leq h$.
- Per ogni nodo x di un RBT T, $n(x) \geq 2^{bh(x)} - 1$ ($n(x)$ è il numero di nodi interni del sottoalbero con radice x).
- Un RBT T con n nodi interni ha altezza $h \leq 2\log_2(n+1)$.

- Ne segue che ogni RBT T con n nodi interni è tale che:

$$h = \Theta(\log n)$$

Recap RBT: Dynamic set operations

- In un RBT T con n nodi interni, le operazioni SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR, richiedono tempo $O(\log n)$.
  - Siccome queste operazioni non modificano l'albero, ed un RBT è (quasi) un BST, possiamo riutilizzare la versione per BST senza modifiche (a parte sostituire NIL con $T.nil$).
  - Il tempo di esecuzione di queste operazioni è $\Theta(h)$ per i BST.
  - Siccome negli RBT si ha $h = \Theta(\log n)$, otteniamo come tempo di esecuzione $\Theta(\log n)$ per gli RBT.
- In un RBT T con n nodi interni, anche le operazioni INSERT e DELETE si possono ottenere con complessità $O(\log n)$.
  - Devono essere modificate rispetto alla versione per BST, in modo da mantenere vere le proprietà RB.
  - Il meccanismo fondamentale è quello delle rotazioni.

Es.1

Vero o falso: dato un RBT T, i sottoalberi con radice in un nodo figlio di $T.root$ sono sempre anch'essi degli RBT.

Es.1

Vero o falso: dato un RBT T, i sottoalberi con radice in un nodo figlio di $T.root$ sono sempre anch'essi degli RBT.

FALSO:

- La radice $T.root$ di un RBT T è nera per definizione.
- E' possibile che un nodo figlio di $T.root$ sia rosso.
- Dunque è possibile che un sottoalbero con radice in un nodo figlio di $T.root$ abbia radice rossa, nel qual caso non è un RBT.

Es.2

Vero o falso: se un nodo x di un RBT T ha un nodo figlio $T.nil$, allora l'altro nodo figlio è anch'esso $T.nil$ oppure è rosso.

Es.2

Vero o falso: se un nodo x di un RBT T ha un nodo figlio $T.nil$, allora l'altro nodo figlio è anch'esso $T.nil$ oppure è rosso.

VERO:

- Supponiamo per assurdo che possa essere anche nero, ovvero supponiamo che possa esistere x tale che

$$x.left = T.nil,\ x.right.color = BLACK$$

- Ma allora esiste un cammino da x a $T.nil$ con un solo nodo nero (il cammino $\{x, x.left = T.nil\}$), ed uno con almeno due nodi neri (qualunque cammino $\{x, x.right, \dots, T.nil\}$).
- Questo è assurdo per la proprietà RB5.

Es.3

Quali sono il minimo e il massimo rapporto tra il numero di nodi rossi e il numero di nodi (interni) neri in un generico RBT?

Es.3

Quali sono il minimo e il massimo rapporto tra il numero di nodi rossi e il numero di nodi (interni) neri in un generico RBT?

Risposta:

- Un RBT può essere totalmente nero, dunque il rapporto minimo tra nodi rossi e neri è 0.
- La proprietà RB4 impone che ogni nodo rosso abbia padre e figli neri. Dunque il massimo numero di nodi rossi si ottiene quando l'RBT è colorato per "strati alternati": per ogni strato nero (a partire dalla radice), il successivo (che contiene il doppio dei nodi) è completamente rosso. Dunque il rapporto massimo tra nodi rossi e neri è 2.

Es.4

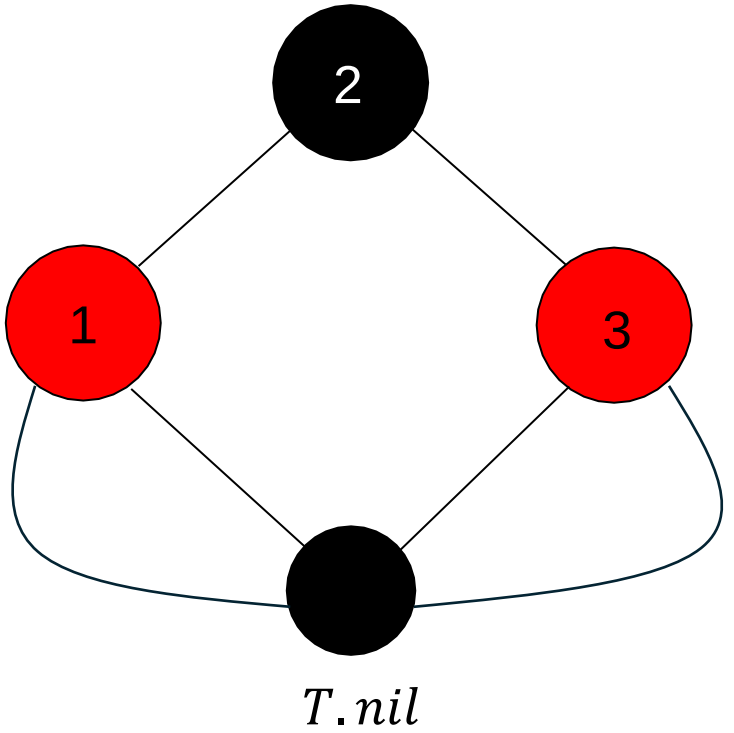
Descrivere una sequenza di invocazioni degli algoritmi `RBT-INSERT(T,z)` ed `RBT-DELETE(T,z)` che produca (a partire da un RBT vuoto) un RBT fatto di esattamente 3 nodi, tutti neri.

Es.4

Descrivere una sequenza di invocazioni degli algoritmi `RBT-INSERT(T,z)` ed `RBT-DELETE(T,z)` che produca (a partire da un RBT vuoto) un RBT fatto di esattamente 3 nodi, tutti neri.

Risposta:

- **`RBT-INSERT(T,2)`**
- **`RBT-INSERT(T,1)`**
- **`RBT-INSERT(T,3)`**
- `RBT-INSERT(T,4)`
- `RBT-DELETE(T,4)`

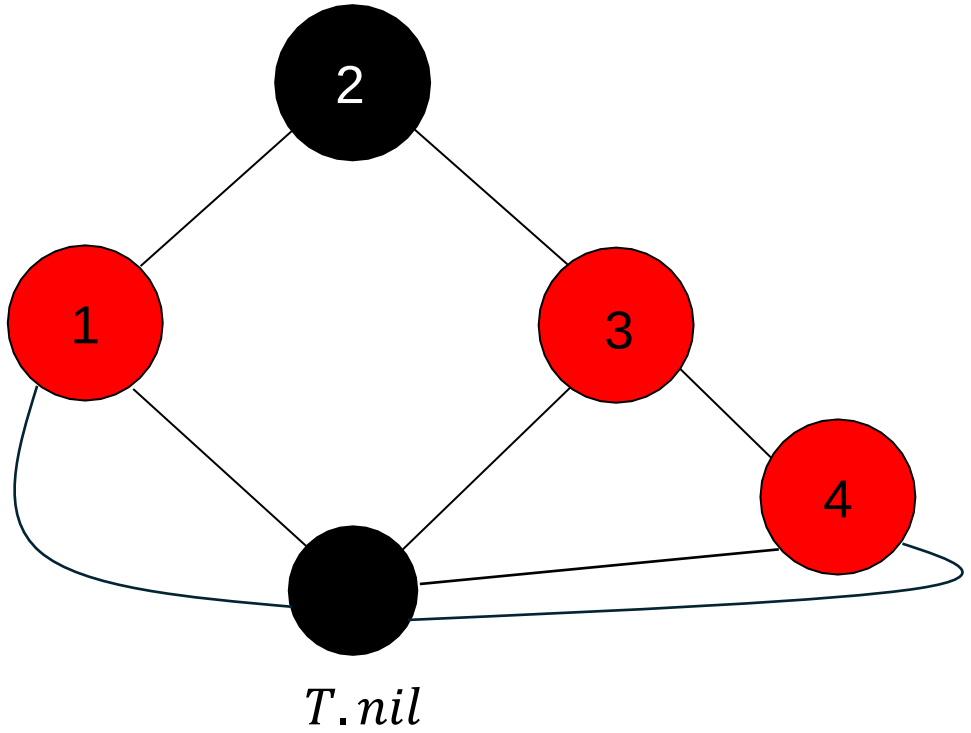


Es.4

Descrivere una sequenza di invocazioni degli algoritmi `RBT-INSERT(T,z)` ed `RBT-DELETE(T,z)` che produca (a partire da un RBT vuoto) un RBT fatto di esattamente 3 nodi, tutti neri.

Risposta:

- `RBT-INSERT(T,2)`
- `RBT-INSERT(T,1)`
- `RBT-INSERT(T,3)`
- **`RBT-INSERT(T,4)`**
- `RBT-DELETE(T,4)`

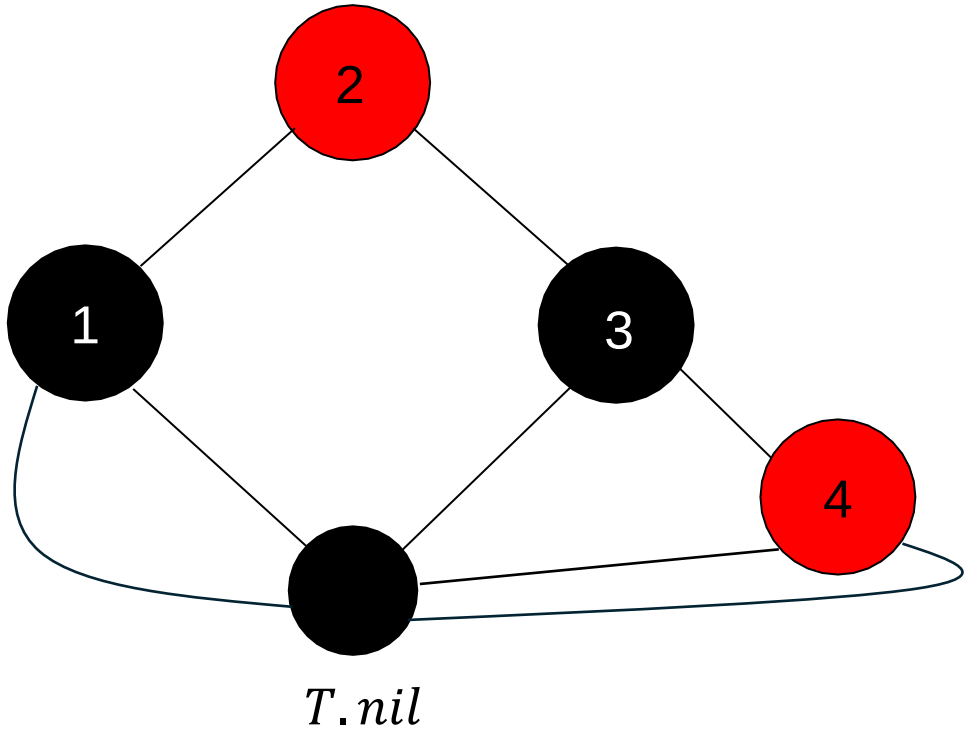


Es.4

Descrivere una sequenza di invocazioni degli algoritmi `RBT-INSERT(T,z)` ed `RBT-DELETE(T,z)` che produca (a partire da un RBT vuoto) un RBT fatto di esattamente 3 nodi, tutti neri.

Risposta:

- `RBT-INSERT(T,2)`
- `RBT-INSERT(T,1)`
- `RBT-INSERT(T,3)`
- **`RBT-INSERT(T,4)`**
- `RBT-DELETE(T,4)`

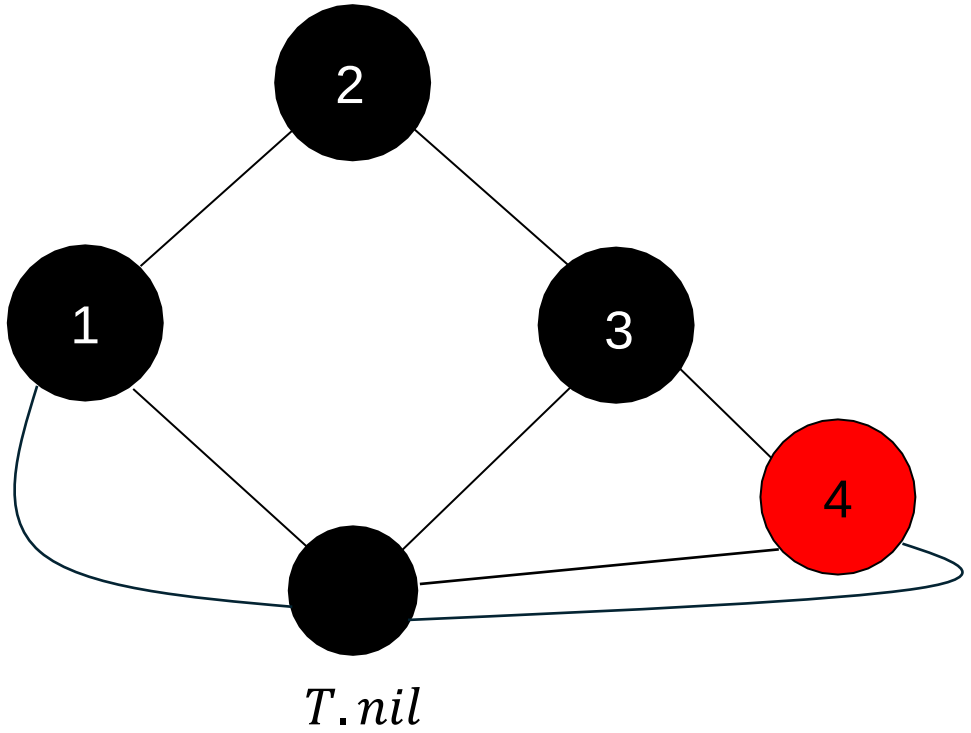


Es.4

Descrivere una sequenza di invocazioni degli algoritmi `RBT-INSERT(T,z)` ed `RBT-DELETE(T,z)` che produca (a partire da un RBT vuoto) un RBT fatto di esattamente 3 nodi, tutti neri.

Risposta:

- `RBT-INSERT(T,2)`
- `RBT-INSERT(T,1)`
- `RBT-INSERT(T,3)`
- **`RBT-INSERT(T,4)`**
- `RBT-DELETE(T,4)`

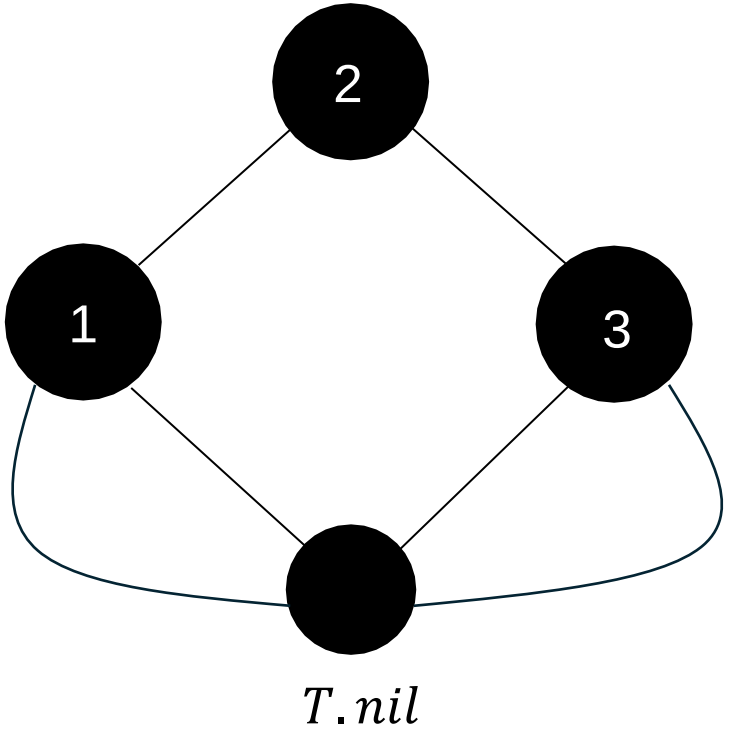


Es.4

Descrivere una sequenza di invocazioni degli algoritmi `RBT-INSERT(T,z)` ed `RBT-DELETE(T,z)` che produca (a partire da un RBT vuoto) un RBT fatto di esattamente 3 nodi, tutti neri.

Risposta:

- `RBT-INSERT(T,2)`
- `RBT-INSERT(T,1)`
- `RBT-INSERT(T,3)`
- `RBT-INSERT(T,4)`
- **`RBT-DELETE(T,4)`**



Es.5

Scrivere un algoritmo che prende in input un BST T i cui nodi hanno un attributo aggiuntivo $color$ (che vale RED oppure $BLACK$) e in cui tutti i puntatori a NIL sono sostituiti con puntatori ad un unico nodo sentinella $T.nil$, e restituisce $TRUE$ se T è un RBT, $FALSE$ altrimenti.

Es.5

Scrivere un algoritmo che prende in input un BST T i cui nodi hanno un attributo aggiuntivo $color$ (che vale RED oppure $BLACK$) e in cui tutti i puntatori a NIL sono sostituiti con puntatori ad un unico nodo sentinella $T.nil$, e restituisce $TRUE$ se T è un RBT, $FALSE$ altrimenti.

Idea:

- L'algoritmo deve solo verificare che le 5 proprietà RB siano rispettate.
- RB1 è garantita per ipotesi.
- RB2 ed RB3 sono immediate da controllare.
- Per verificare che RB4 ed RB5 siano rispettate, l'algoritmo chiama una funzione che ricorsivamente analizza i sottoalberi per calcolare l'altezza nera e controllare che i figli dei nodi rossi siano entrambi neri.

Es.5

```
IS-RBT(T):
 if T.root = T.nil
      return TRUE
 if T.root.color = RED
      return FALSE
 if T.nil.color = RED
      return FALSE
 (bh,is_rb) := CHECK-RB(T.root,0,T)
 return is_rb
```

$\Theta(n)$

```
CHECK-RB(r,bh,T):
 if r = T.nil
       return (bh,TRUE)
 if r.color = RED and
   (r.left.color = RED or r.right.color = red)
       return (bh,FALSE)
 if r.color = BLACK
       bh := bh + 1
 (bh_l,is_rb_left) := CHECK_RB(r.left,bh,T)
 (bh_r,is_rb_right) := CHECK_RB(r.right,bh,T)
 if bh_l ≠ bh_r or
   is_rb_left = FALSE or is_rb_right = FALSE
       return (bh,FALSE)
 return (bh_l,TRUE)
```

Es.6

Scrivere un algoritmo che prende in input un BST T (in cui tutti i puntatori a NIL sono sostituiti con puntatori ad un unico nodo sentinella $T.nil$), e restituisce $TRUE$ se è possibile 'colorare' T in modo da renderlo un RBT, $FALSE$ altrimenti.

Es.6

Scrivere un algoritmo che prende in input un BST T (in cui tutti i puntatori a NIL sono sostituiti con puntatori ad un unico nodo sentinella $T.nil$), e restituisce $TRUE$ se è possibile 'colorare' T in modo da renderlo un RBT, $FALSE$ altrimenti.

Idea:

- Basta verificare che presi due cammini da un qualunque nodo x a $T.nil$, il rapporto tra le loro lunghezze non superi mai 2.
- Per fare ciò, l'algoritmo può ricorsivamente visitare i nodi dell'albero e calcolare per ogni nodo la lunghezza dei cammini minimo e massimo da esso alle $T.nil$ (incrementando di 1 i valori minimo e massimo ottenuti dai figli). Non appena il rapporto tra le due lunghezze eccede 2, restituisce $FALSE$.

Es.6

```
IS-COLOURABLE(T):
 (min, max, ok) := CHECK-HEIGHTS(T.root, T)
 return ok

CHECK-HEIGHTS(r, T):
 if r = T.nil
       return (0, 0, TRUE)
 (min_l, max_l, ok_l) = CHECK-HEIGHTS(r.left, T)
 (min_r, max_r, ok_r) = CHECK-HEIGHTS(r.right, T)
 min := MIN(min_l, min_r) + 1
 max := MAX(max_l, max_r) + 1
 if max > 2 * min
      return (min, max, FALSE)
 return (min, max, ok_l and ok_r)
```

$\Theta(n)$